

Reporte Técnico de Actividades Práctico-Experimentales Nro. 006

1. Datos Generales

Nombre del estudiante(s)	Daniel Saavedra, Emilio Galvez
Asignatura	Estructura de Datos
Ciclo	3er Ciclo
Unidad	2 Unidad
Resultado de aprendizaje de la unidad	Aplica los métodos de ordenación y búsqueda en la resolución de problemas, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Título de la Práctica	Ordenación básica en Java: Burbuja, Selección e Inserción.
Nombre del Docente	Andrés Roberto Navas Castellanos
Fecha	Jueves 20 de noviembre Lunes 24 de noviembre
Horario	07h30 – 10h30 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	5 horas

Link Github: <https://github.com/Dan-San837/Comparacion-de-Ordenacion.git>

2. Objetivo(s) de la Práctica

- Ejecutar y analizar comparativamente los algoritmos de Burbuja, Selección e Inserción sobre casos de prueba, para determinar cuándo conviene cada uno en función de tamaño, grado de orden y duplicados.

3. Materiales y reactivos

- Guía de pruebas con datasets y salidas esperadas.

4. Equipos y herramientas

- JDK OpenJDK (obligatorio).
- IDE: Visual Studio Code (extensión “Extension Pack for Java”) o IntelliJ IDEA Community.
- Sistema de control de versiones: Git; repositorio en GitHub.
- EVA/Moodle institucional: para entrega de evidencias.
- Herramientas de documentación: README Markdown, editor ofimático (Google Docs/LibreOffice/Word).

5. Procedimiento / Metodología Ejecutada

Enfoque metodológico: ABPr (Aprendizaje Basado en Proyectos).

Inicio

- Presentación del objetivo comparativo y criterios de éxito.
- Formación de equipos (3–4) y revisión de la rúbrica.
- Creación de repo Git.
- Lineamientos de uso responsable de IA.

Desarrollo

- Paso 1. Instrumentación (obligatorio)

o Añade contadores a tus algoritmos:

- `comparisons++` al comparar dos claves,
- `swaps++` al intercambiar posiciones.

o Mide tiempo con ``System.nanoTime()`` sin imprimir durante la medición (las trazas distorsionan).

o Ejecuta R repeticiones por caso (sug.: $R=10$), descarta las 3 primeras (calentamiento/JIT) y reporta la mediana de tiempo.

o Aísla IO: carga CSV fuera de la medición; mide sólo el ordenamiento del array en memoria.

- Paso 2. Casos de prueba

o Define clave de orden (p. ej., ``fechaHora`` en ``citas``, ``apellido`` en ``pacientes``, ``stock`` en ``inventario``).

o Convierte a array de la clave (o a registros con ``Comparable`` por clave).

o Ejecuta: Insertion, Selection, Bubble (con “corte temprano” en Burbuja).

o Registra: n , %casi-ordenado, %duplicados, `comparisons`, `swaps`, tiempo(ns) (mediana de R-3 corridas).

- Paso 3. Análisis

o Tablas comparativas por caso (n , orden, duplicados) y gráficos (tiempo vs. n ; tiempo vs. %casi-ordenado).

o Matriz de recomendación (reglas prácticas):

- Casi ordenado + n pequeño/medio $\rightarrow$ Inserción gana (menos movimientos).
- Muchos duplicados $\rightarrow$ Inserción tiende a mantener estabilidad útil; Selección hace $n(n-1)/2$ comparaciones siempre, con pocos swaps.
- Inverso o aleatorio (n pequeño/educativo) $\rightarrow$ cualquiera, pero Burbuja penaliza; Selección constante en comparaciones; Inserción peor en inverso pero mejor si detecta localmente orden.

6. Resultados

Caso 1: Citas Aleatorias (citas_100.csv)

Escenario promedio.

Algoritmo	Comparaciones	Intercambios (Swaps)	Mediana (ns)
Burbuja	4914	2121	170500
Selección	4950	93	105400

Inserción	2218	2121	93900
-----------	------	------	-------

Análisis:

Comparando los tiempos entre los 3 algoritmos, se nota que el algoritmo de selección tuvo un menor tiempo comparado con los otros algoritmos, ya que realizó menos intercambios.

```
=== Citas (Aleatorio) ===
```

Algoritmo	Comparisons	Swaps	Mediana(ns)
Insertion	2218	2121	93900
Selection	4950	93	105400
Bubble	4914	2121	170500

Caso 2: Casi Ordenadas (citas_100_casi_ordenadas.csv)

Mejor caso simulado.

Algoritmo	Comparaciones	Intercambios (Swaps)	Mediana(ns)
Burbuja	4170	243	49800
Selección	4950	5	41400
Inserción	342	243	10500

Análisis:

En este caso, podemos ver que Insercion ganó mucho, esto debido a que, al tener los datos casi ordenados, trabaja casi en tiempo lineal, por otra parte Selección sigue realizando todas las comparaciones ciegamente.

```
=== Citas (Casi Ordenadas) ===
```

Algoritmo	Comparisons	Swaps	Mediana(ns)
Insertion	342	243	10500
Selection	4950	5	41400
Bubble	4170	243	49800

Caso 3: Pacientes con Duplicados (pacientes_500.csv)

Prueba de estabilidad.

Algoritmo	Comparaciones	Intercambios (Swaps)	Mediana(ns)
-----------	---------------	----------------------	-------------



Burbuja	123970	59569	5714200
Selección	124750	491	769800
Inserción	60064	59569	376800

Análisis:

Podemos ver que el algoritmo con menor número de comparaciones y tiempo es Inserción, en cambio, Selección tiene el menor número de iteraciones

```

=== Pacientes (Duplicados/Estabilidad) ===
Algoritmo  Comparisons  Swaps  Mediana(ns)
Insertion   60064        59569   376800
Selection   124750        491     769800
Bubble      123970        59569   5714200
  
```

Caso 4: Inventario Inverso (inventario_500_inverso.csv)

Peor caso. Orden descendente estricto.

Algoritmo	Comparaciones	Intercambios (Swaps)	Mediana(ns)
Burbuja	124750	124750	926500
Selección	124750	250	374100
Inserción	124750	124750	1107700

Análisis:

Burbuja e Inserción sufren mucho aquí, podemos ver que tienen muy mal tiempo, además que, el número de comparaciones no varía entre los tres algoritmos.

```

=== Inventario (Inverso) ===
Algoritmo  Comparisons  Swaps  Mediana(ns)
Insertion   124750        124750  1107700
Selection   124750         250     374100
Bubble      124750        124750   926500
  
```

7. Preguntas de Control

- ¿Por qué imprimir trazas durante la medición distorsiona los tiempos?

Porque la operación de salida es demasiado lenta comparada con las instrucciones del algoritmo y al final se termina midiendo el rendimiento de la consola, pero no la del algoritmo completo.

- **Explica por qué Selección tiene comparaciones $n(n-1)/2$ sin importar el orden inicial.**

Porque el algoritmo no tiene forma de saber que los datos ya están ordenados al inicio, es decir, siempre debe verificar todas las opciones posibles para confirmar cuál es el mínimo en cada sublista. Esto se debe a la forma en que funciona su proceso de búsqueda del mínimo en cada pasada.

- **¿Por qué la Inserción es competitiva en datos casi ordenados?**

Porque este algoritmo aprovecha el hecho de que los elementos ya están cerca de su posición correcta, gracias a que los desplazamientos necesarios son mínimos y las comparaciones se reducen mucho.

- **¿Qué papel juegan los duplicados en la estabilidad del resultado?**

Los duplicados son clave porque permiten verificar si un algoritmo mantiene el orden original entre los elementos iguales., en donde sí se conserva ese orden, el algoritmo es estable y si lo altera es inestable.

- **¿Por qué Burbuja con corte temprano mejora en “casi ordenado” pero no en “inverso”?**

Porque el corte temprano solo se activa cuando no hubo intercambios en una pasada, es decir en una lista casi ordenada eso ocurre rápido, así que el algoritmo se detiene antes, mientras que en una lista inversa siempre hay intercambios en cada pasada, así que nunca se cumple la condición de corte y el algoritmo recorre todo, de la misma forma que en su peor caso.

8. Conclusiones

- Hablando acerca de la eficiencia de cada algoritmo de ordenación realizado con las pruebas de datasets en esta práctica, tenemos que:
 - **Sobre Inserción:** Este algoritmo es ideal cuando sabemos que los datos ya vienen ordenados en cierta medida y su capacidad adaptativa es lo que lo hace muy superior en este aspecto.
 - **Sobre Selección:** Es consistente pero rígido ya que no importa si los datos están ordenados o desordenados porque siempre tardará lo mismo.
 - **Sobre Burbuja:** En general es el menos eficiente sobre todo en listas inversas, por lo cual debería evitarse para datasets grandes a menos que sea con fines educativos.

9. Bibliografía / Referencias

- [1] OpenDSA Project, “Sorting and Searching Modules,” Virginia Tech, 2021–2024 (REA con visualizaciones y ejercicios).
- [2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.
- [3] Oracle, “Java SE 17–21 Documentation: `Arrays`, Collections, and I/O (`java.nio.file`), and benchmarking notes,” 2021–2025.
- [4] OpenJDK, “JMH – Java Microbenchmark Harness: Samples and Guidance,” 2020–2025 (guía práctica de mediciones reproducibles).